ROAD DETECTAI

AI-Powered Pothole Detection System

Project Implementation and Results Report

Technology: Python • YOLOv8 • Streamlit
Model: YOLOv8n | Training device: CPU

Prepared by: Manoj Kumar Mandal

Executive Summary

Road DetectAI is a computer-vision prototype that detects potholes in uploaded road images. It uses a YOLOv8n object-detection model trained on a pothole dataset and a Streamlit interface to make predictions accessible through a web browser. The dashboard displays annotated images, confidence scores, bounding-box coordinates, detection counts, and an estimated severity category. Users can download a CSV of detection details and an annotated PNG image.

In the reported validation run, the model was evaluated on 133 images containing 330 pothole instances. The reported metrics were precision 0.762, recall 0.633, mAP@50 0.721, and mAP@50–95 0.437. These results indicate useful detection capability, but also show that some potholes may be missed. They are validation results, not a guarantee of performance in every real-world setting.

1. Problem Statement

Potholes can create road-safety risks and require inspection and maintenance. Manually reviewing large numbers of road images can take time. This project explores an AI-assisted approach that automatically identifies likely potholes in images and presents the results in an easy-to-use interface.

2. Project Objectives

• Train a YOLOv8n object-detection model to identify potholes.

• Allow a user to upload a road image through a Streamlit dashboard.

• Display detected potholes using bounding boxes and confidence scores.

• Calculate an estimated severity category from predicted bounding-box area coverage.

• Show a detection summary and detailed coordinates.

• Allow users to download detection details as CSV and the annotated image as PNG.

• Evaluate the trained model using standard object-detection metrics.

3. Technologies and Tools

Technology	Role in the project
Python	Main programming language for training, inference, and application logic.
Ultralytics YOLOv8n	Object-detection model used to locate potholes.
Streamlit	Web interface for image upload, controls, and results.
Pandas	Creates the detection-details table and CSV export.
Pillow (PIL)	Reads uploaded images and prepares images for display/download.
OpenCV / YOLO plotting	Supports image processing and visualization of predicted boxes.

4. Dataset and Configuration

The training script reads the dataset configuration from Pothole.v1-raw.yolov8/data.yaml. The configuration defines one class, pothole, with training, validation, and test image paths. The completed run scanned 465 training images and 133 validation images, with 330 annotated pothole instances in the validation set.

The dataset configuration identifies the source license as ODbL v1.0. Before redistributing the dataset, verify the original source terms and attribution requirements.

5. System Architecture

1. User uploads a JPG, JPEG, PNG, or WEBP road image.

2. The app opens the image and converts it to RGB format.

3. The trained YOLOv8n model predicts pothole bounding boxes and confidence scores.

4. The app collects class labels, confidence values, and box coordinates (X1, Y1, X2, Y2).

5. The severity helper calculates the total predicted box area as a percentage of the image area.

6. The dashboard displays the annotated image, detection count, coverage, severity, and detail table.

7. The user can download the table as CSV and the annotated image as PNG.

6. Step-by-Step Implementation

Step 1 — Prepare the project environment

Create a project folder and a Python virtual environment. Install the packages listed in requirements.txt (Streamlit, Ultralytics, Pandas, Pillow, and OpenCV-headless). A virtual environment isolates Python packages; it does not make CPU training faster.

Example commands (Windows PowerShell):

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Step 2 — Configure the dataset

Place the dataset in the expected folder and check the paths in Pothole.v1-raw.yolov8/data.yaml. The training log confirmed that 465 training images and 133 validation images were found. Confirm that the paths point to the intended pothole dataset before training.

Step 3 — Train the YOLOv8n model

The train.py script loads pretrained yolov8n.pt weights and fine-tunes the detector on the pothole dataset. The completed configuration used 30 epochs, image size 416, batch size 2, zero data-loader workers, CPU execution, and early-stopping patience of 8 epochs. Training took approximately 0.949 hours on the reported Intel Core i3-1005G1 CPU.

The resulting checkpoint was saved at runs/detect/runs/pothole/pothole_img416_fixed/weights/best.pt. The application must point to this file to use the new model.

Step 4 — Implement the severity calculation

The severity.py function receives predicted bounding boxes and image width and height. It calculates each box's width and height, sums the box areas, divides by total image area, and multiplies by 100. It assigns a category using the configured thresholds:

Condition	Category
No detections	NONE
Coverage < 3%	LOW
3% ≤ coverage < 10%	MEDIUM
10% ≤ coverage < 20%	HIGH
Coverage ≥ 20%	CRITICAL

This is a visual heuristic, not a physical measurement of pothole size or a professional road-condition rating. Overlapping boxes can count the same image area more than once.

Step 5 — Build the Streamlit application

app.py sets the page layout and styling, provides sidebar navigation, and loads the trained model with Streamlit resource caching. The dashboard accepts an image, lets the user adjust the confidence threshold, runs inference on the CPU, and displays the annotated result. It also displays detection details in a Pandas table and provides CSV and PNG downloads.

The shared app.py code uses imgsz=320 for inference while the model was trained at 416. YOLO can infer at a different size, but for a consistent first comparison, set imgsz=416 in model.predict() and update the Model Information text to match.

Step 6 — Run and test the application

Run: streamlit run app.py. Open the local URL printed in the terminal. Upload road images, run detection, review boxes and confidence scores, and download the results. Test multiple images, including small potholes, distant potholes, shadows, and images with no potholes. Record missed detections and false

positives.

7. Model Evaluation and Results

The completed training log reported these validation metrics for best.pt. The validation set contained 133 images and 330 pothole instances.

Metric	Value	Interpretation
Precision	0.762 (76.2%)	Of predicted detections, the proportion that were correct under the evaluation settings.
Recall	0.633 (63.3%)	The proportion of annotated potholes detected by the model.
mAP@50	0.721 (72.1%)	Mean average precision at IoU threshold 0.50.
mAP@50–95	0.437 (43.7%)	Mean average precision averaged across IoU thresholds 0.50 to 0.95.

Precision is higher than recall, suggesting some actual potholes are missed even though many predicted boxes are correct. The mAP@50–95 value is lower than mAP@50 because stricter overlap thresholds require more accurate box placement. These metrics should not be described as general classification accuracy.

The earlier model yolov8n-2 reported precision 0.674, recall 0.606, mAP@50 0.631, and mAP@50–95 0.365. The new run's reported values are higher on all four metrics. For a fair comparison, ensure both models were evaluated on the same validation dataset and settings.

8. Results Produced by the Application

• Annotated road image with predicted pothole boxes.

• Count of detected potholes in the uploaded image.

• Confidence score and bounding-box coordinates for each detection.

• Estimated image-area coverage and severity category.

• Downloadable CSV report.

• Downloadable annotated PNG image.

9. Failure Cases and Limitations

• Small or distant potholes may be missed because they occupy few pixels.

• Low light, shadows, rain, blur, or partial obstruction may make potholes harder to detect.

• Road patches, cracks, dark markings, and shadows may be mistaken for potholes.

• Lowering the confidence threshold may recover detections but can also increase false positives.

• Image-area coverage depends on camera angle, image crop, and predicted boxes; it is not physical pothole area.

For a formal report, attach actual failure images and explain the model prediction versus the correct outcome. Specific failure images were not provided for this report.

10. Future Improvements

• Evaluate the final model on a held-out test set.

• Save and include a confusion matrix and precision–recall curve where available.

• Test different confidence thresholds and image sizes on the same evaluation data.

• Improve recall while monitoring false positives.

• Add example images and clear installation instructions to the repository.

• Deploy to a suitable hosting platform after checking model and dataset licensing.

11. Reproducibility and Repository Notes

The main files are app.py, train.py, severity.py, and requirements.txt. Ensure the trained best.pt file is available at the path configured in app.py. The exact folder layout may vary. For GitHub, include a .gitignore, avoid committing the virtual environment, cache files, and large generated training outputs, and verify dataset and model redistribution permissions.

12. Conclusion

Road DetectAI demonstrates an end-to-end pothole detection workflow: dataset configuration, YOLOv8n training, model inference, severity estimation, and a Streamlit interface for reviewing and exporting predictions. The latest reported validation metrics are precision 76.2%, recall 63.3%, mAP@50 72.1%, and mAP@50–95 43.7%. The results are promising for a prototype, while the recall and known visual limitations show that additional testing and improvement are needed before real-world operational use.

Appendix: Quick Start

1. Install Python and dependencies from requirements.txt.
2. Confirm best.pt exists and WEIGHTS in app.py points to it.
3. Run streamlit run app.py.
4. Upload a road image and click Run Road Damage Detection.
5. Review detections, severity estimate, and download the CSV or annotated image.